

When the bigER → SQL generator translates an `.er` model into DDL, several gaps in the source notation force the emitter to make choices. The notation says "this is a string"; SQL needs `VARCHAR(80)`. The notation says `Author[1] -> Book[N]`; SQL has no native "cardinality" — only PK / FK / UNIQUE / NOT NULL. This file lists every such choice, why it was made, and where in the code it lives.

All file/line references point at the emitter on this branch:
[packages/language-server/src/export/sql/](https://github.com/erlang/otp/blob/master/packages/language-server/src/export/sql/).

1. Datatype mapping

Where: `dialects.ts`, `renderDatatype` in the emitter.

Assumptions:

- bigER datatypes are not restricted by the grammar — any identifier works (`int` , `INT` , `INTEGER` , `string` , `DATETIME` , `FOOBAR`). The emitter asks the active `Dialect` to map the input to a dialect-native name.
- Each dialect declares "first-of-category" types. Anything mapped into category `INTEGER` resolves to that dialect's first `INTEGER` type (postgres → `BIGINT` ; mysql → `BIGINT` too). Same for `FLOAT` , `DECIMAL` , `VARCHAR` , `CHAR` , `DATE` , `DATE_TIME` , `BLOB` , `CLOB` , `BOOLEAN` .
- bigER `string` → dialect's first `CHARACTER` type (postgres `VARCHAR` , mysql `VARCHAR`). Sizeless. MySQL will accept this; some MySQL versions may reject `VARCHAR` without a size — if you hit that, declare a size.

2. Nullability — NOT NULL by default

Where: `renderAttribute` in `ddl-emitter.ts`.

Rule: every column gets `NOT NULL` unless it's part of the PRIMARY KEY (in which case the PK clause implies NOT NULL) or the attribute is marked `optional`.

Why default-NOT-NULL and not default-nullable:

- Matches typical DDL convention — `NOT NULL` is the well-known default recommendation in most style guides.
- `optional` in the grammar reads as the explicit annotation. It's clearer to mark the exception than to mark the rule.
- The alternative (default-nullable, NOT NULL only when cardinality is 1) would mean every entity attribute is nullable unless explicitly `cardinality-constrained` — surprising and lossy

3. Cardinality — **UNIQUE** from look-across constraints

Where: `emitRelationship` and `isAtMostOne` at the bottom of `ddl-emitter.ts`.

Interpretation: bigER uses Chen-style "look-across" cardinality. In

$A[m..n] \rightarrow B[p..q]$, the $m..n$ describes A's participation per instance of B, and $p..q$ describes B's participation per instance of A. So:

- $\text{Author}[1] \rightarrow \text{Book}[N]$ means "each Book has 1 Author; each Author has N Books." Each Book appears at most once in the bridge $\rightarrow$ **UNIQUE** on the bridge's `book_id` column.
- $\text{Book}[N] \rightarrow \text{Tag}[N]$ means "each Tag has many Books; each Book has many Tags." Neither side is **UNIQUE**.
- $\text{Husband}[1] \rightarrow \text{Wife}[1]$ means each side participates once $\rightarrow$ both FK columns are **UNIQUE**.

Encoding rule: a participant's FK column in the bridge is **UNIQUE** iff

4. Aggregation and composition

Where: `emitRelationship`, the `isWhole` branch.

Assumptions:

- **Composition** (`*-` / `-*`): the side with the `*` is the *whole*.
Bridge → whole FK gets `ON DELETE CASCADE`. Deleting the whole removes the relationship rows referencing it, matching the "part's lifecycle is bound to the whole" semantics.
- **Aggregation** (`o-` / `-o`): plain FK, no CASCADE. SQL has no built-in notion of "aggregation" beyond foreign-key reference, and the part can outlive the whole.
- Only **binary** composition is honoured. For ternary+ the type-decorator is ignored and a plain FK is emitted (the validator already warns about ternary aggregation/composition under UML at

5. Weak entity → folded into one table with owner FK

Where: `emitWeakEntity` and `findWeakOwner`.

Assumptions:

- A weak entity is identified by the first `weak relationship` in the model that mentions it. The first non-weak participant of that relationship is taken as the owner.
- The owner's PK columns are inlined into the weak entity's table. The combined PK is `(owner_pk_cols u partial_key_cols)`. An FK with `ON DELETE CASCADE` from the inlined owner-PK columns points back at the owner — deleting the owner removes its weak rows.
- The `weak relationship` itself does NOT emit a separate bridge table (the FK is already in the weak entity's table). This is enforced at `emit` via

6. Generalization — class-table inheritance

Where: `emitInheritedEntity`.

Strategy chosen: class-table inheritance. The subclass table duplicates the parent's PK columns as its own PK and points an FK at the parent. The subclass row's existence is bound to the parent row. The subclass table holds only its own (non-derived, non-multivalued) attributes — it does NOT copy parent's non-key attrs.

Why class-table over alternatives:

- Single-table (everything in one wide table with a discriminator) loses the entity boundary and bloats the parent table; awkward when subclasses have many type-specific attrs.
- Concrete-table (each subclass flattens parent attrs in) requires marking parents as abstract — bigER has no such marker — and duplicates schema.

7. Multi-valued attribute → side table

Where: `emitMultivaluedTable` .

Assumptions:

- An attribute marked `multivalued` is dropped from the entity's main table (excluded by `emittableAttrs`) and emitted as a side table named `<Entity>_<attr>` .
- Side table schema: parent's PK columns + the multivalued attribute column; composite PK across all of them; `ON DELETE CASCADE` FK back to parent. The multivalued column is in the PK, so it's implicitly NOT NULL and duplicate values per parent are rejected.
- Side table names are unquoted concatenation. Collisions (e.g. two entities both named `Item` with multivalued `tag`) would produce two tables with the same name. Not handled.

8. Roles → FK column-name prefix for self-references

Where: `emitRelationship`, the `hasDuplicates` block.

Assumptions:

- A role label is used as an FK-column-name prefix *only* when the same entity appears more than once in the relationship (self-reference, or a pair of participants of the same entity). In other cases the role is parsed but doesn't affect emission.
- The column-name pattern is `<role>_<keyName>` — e.g. `manager_id` and `reports_id` for `Employee[1|"manager"] -> Employee[N|"reports"]`.

Limitations:

- If a relationship has duplicated participants but no roles, FK columns

9. `generatedDrop` flag → reverse-emission DROP order

Where: `emit` with `opts.generatedDrop`.

Assumptions:

- When `opts.generatedDrop` is true, the emitter prepends `DROP TABLE IF EXISTS <name>;` for every table the run will create. The drops appear in **reverse emission order**.
- Reverse-emission order is correct because every FK reference points "upward":
 - Bridge tables FK to entity tables → bridge tables emit after entities.
 - Subclass tables FK to parent tables → subclasses extend their parent declared earlier in the source.
 - Multivalued side tables FK to their owner → emitted in the same loop iteration as their owner (right after).

10. Things we deliberately do NOT generate

These are valid bigER inputs that produce no SQL artefact and no warning. Document so the engineer doesn't think they're bugs:

- `derived` attributes — by definition computed from other columns; no storage. Skipped at `emittableAttrs`.
- `aggregation` operators (`o-` / `-o`) decoration — emitted as plain FK. SQL has no semantic distinction.
- Visibility modifiers (`public` , `private` , `+` , `-` , etc.) — UML-only surface concern; ignored for SQL.
- `notation = chen|bachman|...` declarations — affect diagram rendering only.
- The model's name (`erdiagram MyModel`) — not used in any table or schema name. SQL output has no enclosing namespace.

11. Known limitations affecting all generated SQL

These bite real users; flagged here so they're not lost.

- **Reserved-word collision:** entity named `Order`, `User`, `Group`, `Date`, etc. produces SQL that fails to execute. The emitter never quotes identifiers. Per-dialect quoting (e.g. `"Order"` in postgres, ``Order`` in mysql) is a follow-up.
- **Anonymous constraints:** FK and UNIQUE constraints are emitted without `CONSTRAINT <name>` clauses. DB-level violation messages reference auto-generated names instead of the relationship name, hurting diagnosability. Adding `CONSTRAINT fk_<rel>_<role_or_target>` is a small follow-up.
- **No indexes:** only PKs and the constraints above get indexes (typically auto-created). FK columns are not explicitly indexed; on MySQL, this is usually the first FK constraint in a table.

12. Maintaining the goldens

When you change the emitter:

1. Run `REGEN_GOLDENS=1 yarn vitest run test/regen-fixtures.test.ts` from `packages/language-server/`. This rewrites every `<stem>.{postgres,mysql}.sql` fixture from the corresponding `.er`.
2. Review the diffs.
3. Run `SNAPSHOT_SHAPES=1 yarn test` to refresh the matching `<stem>.<dialect>.shape.json` sidecars. Review those diffs too.
4. Then run `yarn test` to confirm Stages 0-5 are green.

The regen helper lives at

[test/regen-fixtures.test.ts](#);

it's a vitest spec gated on the `REGEN_GOLDENS=1` env var and skipped otherwise.